

SérieBox

Sistema Web de Cadastro e Consulta de Series

Atividade Avaliativa Final — 5,0 pontos
Disciplina: Desenvolvimento Web com React
UniCesumar — Campus Ponta Grossa
Data de entrega: 16/06/2025

1. Introducao

Este relatorio descreve o desenvolvimento do **SérieBox**, um sistema web para cadastro e consulta de series de televisao, criado como Atividade Avaliativa Final da disciplina de Desenvolvimento Web com React, na UniCesumar — Campus Ponta Grossa.

A proposta foi planejar, implementar e apresentar uma aplicacao web completa, cobrindo desde a estrutura semantica em HTML5 ate a componentizacao em React, passando por estilizacao responsiva, interatividade com JavaScript, persistencia de dados no navegador e consumo de uma API externa.

O sistema permite que o usuario **busque series em tempo real** via API publica do TVMaze, **cadastre series manualmente** com validacao de formulario, **gerencie sua lista pessoal** com filtros e alteracao de status, e visualize **detalhes completos** de cada titulo.

2. Funcionalidades da Aplicacao

- **Busca em tempo real:** O usuario digita o nome de uma serie e o sistema faz requisicao a API TVMaze, exibindo resultados em cards com poster, nota e genero.
- **Adicionar a lista:** A partir dos resultados da busca, qualquer serie pode ser adicionada a lista pessoal com um clique, com status inicial 'Quero assistir'.
- **Cadastro manual:** Formulario com campos de nome, genero, status, nota (0-10) e comentario pessoal para adicionar series sem depender da API.
- **Validacao de formulario:** Campos obrigatorios sao verificados antes do envio; mensagens de erro aparecem abaixo de cada campo com problema.
- **Minha Lista:** Tela de consulta com todas as series salvas, filtro por status e busca por nome.
- **Alteracao de status:** O usuario pode mudar o status diretamente na lista, sem abrir formulario separado.
- **Pagina de Detalhes:** Rota dinamica /serie/:id que busca e exibe sinopse, nota, generos, data de estreia e status.
- **Persistencia de dados:** A lista pessoal e salva no LocalStorage do navegador, permanecendo mesmo apos fechar o browser.

3. Tecnologias Utilizadas

Tecnologia	Versao	Finalidade
React	18.3	Biblioteca principal (componentes, estado, ciclo de vida)
React Router	6.26	Roteamento entre paginas (Home, Cadastro, Lista, Detalhes)
Tailwind CSS	4.0	Estilizacao utilitaria, responsividade e Design System
Vite	5.4	Ambiente de desenvolvimento e build de producao
Fetch API	nativa	Requisicoes HTTP a API externa TVMaze (JSON)
LocalStorage	nativa	Persistencia dos dados da lista pessoal no navegador
TVMaze API	publica	Fonte de dados de series (sem necessidade de chave de acesso)

Git / GitHub	—	Controle de versao e publicacao do codigo-fonte
--------------	---	---

4. Estrutura HTML e do Projeto

4.1 Estrutura de Arquivos

Separacao de responsabilidades com cada camada em seu proprio diretorio:

- **src/api/tvmaze.js** — Requisicoes a API externa com Fetch
- **src/hooks/useLocalStorage.js** — Hook customizado que sincroniza estado React com LocalStorage
- **src/context/ListaContext.jsx** — Contexto global com funcoes adicionar, editar e remover
- **src/components/** — Componentes reutilizaveis: Header, SerieCard, Listaltem, Ui
- **src/pages/Home.jsx** — Busca de series via API — exhibe resultados em cards
- **src/pages/Cadastro.jsx** — Formulario com validacao para cadastro manual
- **src/pages/MinhaLista.jsx** — Lista pessoal com filtro por status e busca por nome
- **src/pages/Detalhes.jsx** — Rota dinamica /serie/:id com detalhes completos

4.2 Estrutura HTML e Semantica

O index.html segue HTML5 semantico com meta charset, viewport e ponto de montagem do React. As paginas usam header, main, nav, article, section e footer para garantir acessibilidade e boa estrutura de documento.

5. Layout, Estilo e Responsividade

5.1 Estrategias de Layout

- **Flexbox:** Header, cards de lista e botoes de filtro.
- **CSS Grid:** grade de resultados com 2 colunas no mobile, 3 no tablet e 4 no desktop.
- **Box Model:** controlado via classes de padding, margin e border do Tailwind.
- **Sticky header:** cabecalho fixo no topo com backdrop blur.

5.2 Responsividade com Media Queries

Os prefixos do Tailwind (sm:, lg:) geram Media Queries automaticamente. A classe grid-cols-2 sm:grid-cols-3 lg:grid-cols-4 adapta a grade sem nenhuma Media Query escrita manualmente.

5.3 Design System e Componentizacao

Paleta consistente (fundo #0e1116, destaque amber #f59e0b), tipografia padronizada, border-radius uniforme e estados de hover/focus definidos para todos os elementos interativos, seguindo principios de Design System.

6. Interatividade e Logica de Programacao

6.1 Fetch API e Manipulacao de JSON

O modulo src/api/tvmaze.js contem duas funcoes assincronas que requisitam a API publica do TVMaze. A resposta JSON e manipulada para extrair os campos necessarios. O tratamento via try/catch exhibe falhas de rede de forma amigavel.

6.2 Validacao de Formulario

- Campo **Nome:** obrigatorio, minimo 2 caracteres.
- Campo **Nota:** numero entre 0 e 10 (aceita decimais).
- Mensagens de erro abaixo do campo correspondente.
- Atributo **noValidate** desativa a validacao nativa do browser.

6.3 LocalStorage e Eventos

O hook useLocalStorage le o valor salvo no primeiro render e grava o novo valor a cada mudanca. Eventos gerenciados pelo React: onSubmit com preventDefault, onChange nos inputs e onClick nos botoes de filtro e remover.

7. Desenvolvimento com React

7.1 Componentes e Props

- **SerieCard**: recebe 'serie' e 'onAdicionar' — card de resultado de busca.
- **ListItem**: recebe 'serie', 'onRemover' e 'onEditarStatus' — item da lista.
- **Loader**, **Mensagem**, **StatusBadge**: componentes de UI genericos reutilizados.
- **Header**: NavLink destaca automaticamente a rota ativa.

7.2 useState e useEffect

useState controla todos os estados locais: busca, resultados, carregamento, erros e campos do formulario. useEffect dispara a busca na pagina de Detalhes sempre que o parametro de rota (:id) muda, com funcao de limpeza para cancelar atualizacoes em componentes ja desmontados.

7.3 React Router — Rotas da aplicacao

Rota	Componente	Descricao
/	Home.jsx	Busca de series via API
/cadastro	Cadastro.jsx	Formulario de cadastro manual
/minha-lista	MinhaLista.jsx	Lista pessoal com filtros
/serie/:id	Detalhes.jsx	Detalhes dinamicos por ID
*	NaoEncontrada	Pagina 404

7.4 DOM Virtual e Ciclo de Vida

O React atualiza apenas os nos do DOM que realmente mudaram. Quando o usuario altera o status de uma serie, so o ListItem correspondente e re-renderizado. A funcao de limpeza do useEffect funciona como componentWillUnmount.

8. Boas Praticas

8.1 Git e README

- Projeto versionado com Git e publicado no GitHub.
- .gitignore exclui node_modules e arquivos de build.
- README.md completo com descricao, tecnologias, estrutura e instrucoes (npm install + npm run dev).

8.2 Outras Praticas

- **Separacao de responsabilidades:** API, hooks, contexto, componentes e paginas em pastas distintas.
- **Acessibilidade:** tags semanticas, atributos alt em imagens e foco visivel.
- **Prefers-reduced-motion:** animacoes desativadas para usuarios que preferem menos movimento.
- **Tratamento de erros:** try/catch em todas as chamadas a API com feedback visual.
- **Lazy loading:** imagens carregam com loading='lazy' para melhor desempenho.

9. Desafios Encontrados e Solucoes

Configuracao do Tailwind CSS v4

A versao 4 eliminou o tailwind.config.js. Solucao: plugin @tailwindcss/vite e @import 'tailwindcss' no CSS.

Compartilhamento de estado entre paginas

A lista precisava estar disponivel em todas as rotas. Solucao: ListaContext com Provider no topo da arvore.

Cancelamento de efeitos assincronos

Fetch em andamento ao navegar causaria setState em componente desmontado. Solucao: variavel 'ativo' na limpeza do useEffect.

HTML embutido na resposta da API

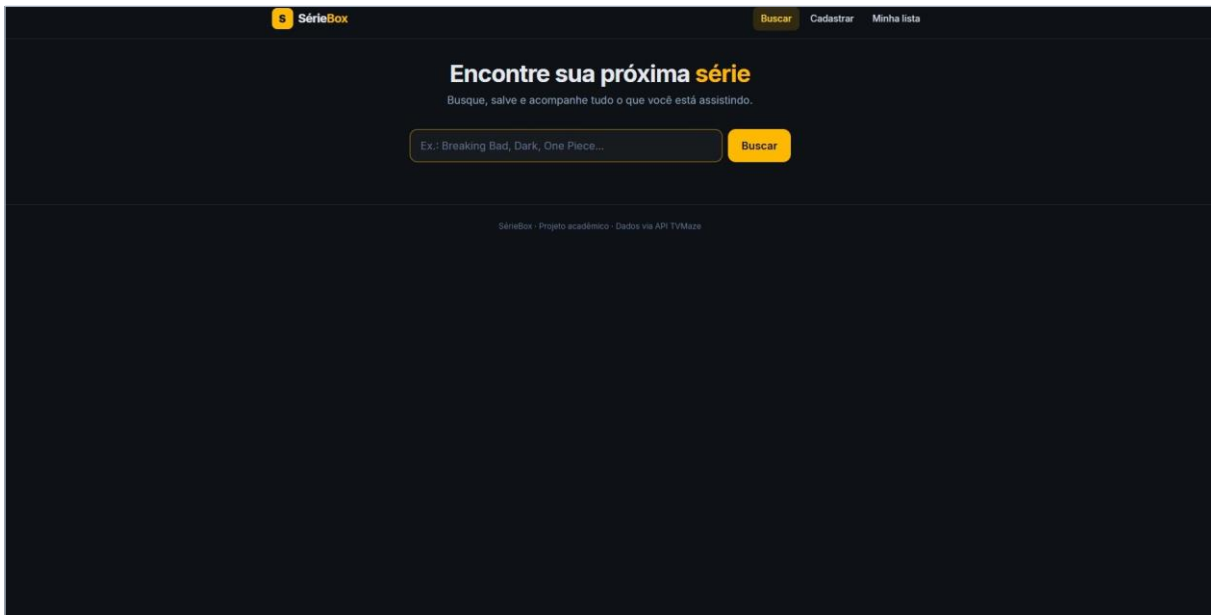
O campo 'summary' da TVMaze vem com tags HTML. Solucao: replace(/<[>]+>/g, "") para limpar o texto.

Geracao de IDs unicos

crypto.randomUUID() com fallback para Date.now() + Math.random() em ambientes sem suporte.

10. Capturas de Tela das Principais Interfaces

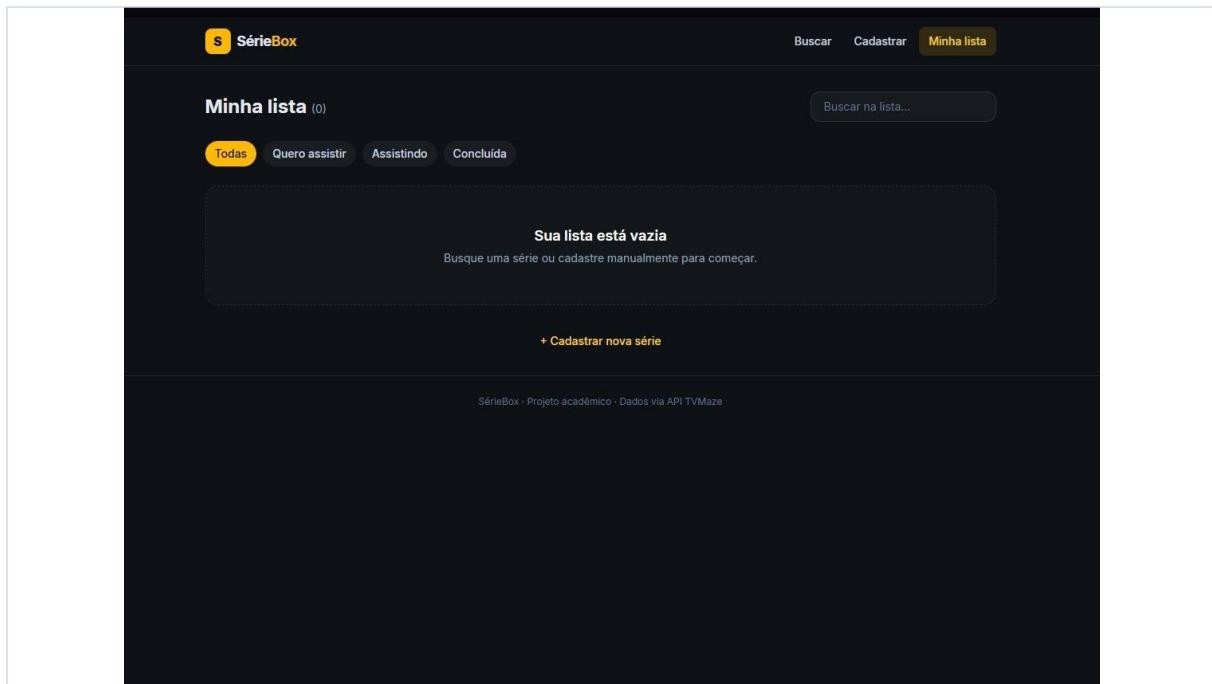
As imagens abaixo mostram as quatro telas principais da aplicação SérieBox em funcionamento:



Tela 1 — Pagina de Busca: campo de pesquisa e resultados retornados pela API TVMaze.

O formulário de cadastro de série, intitulado 'Cadastrar série', está sobreposto ao mesmo cabeçalho da interface de busca. O subtítulo indica 'Adicione manualmente uma série à sua lista pessoal.' O formulário contém os seguintes campos: 'Nome *' com o exemplo 'Ex.: The Office', 'Gênero' com o exemplo 'Ex.: Comédia, Drama', 'Status' com um menu suspenso selecionando 'Quero assistir', e 'Nota (0-10)' com um campo 'Opcional'. Abaixo desses campos, há um campo de texto para 'Comentário' com o placeholder 'O que achou? Anotações...'. Um botão amarelo 'Salvar na minha lista' está posicionado na base do formulário. O rodapé da página também contém o texto 'SérieBox - Projeto acadêmico - Dados via API TVMaze'.

Tela 2 — Formulário de Cadastro: campos com validação para adicionar séries manualmente.



Tela 3 — Minha Lista: filtros por status, busca por nome e opções de remover/editar.



Tela 4 — Página de Detalhes: sinopse, nota, gêneros e data de estreia via rota /serie/:id.

11. Conclusão

O desenvolvimento do SérieBox permitiu aplicar na prática todos os conceitos da disciplina: HTML5 semântico, Tailwind CSS (Flexbox, Grid, Media Queries), JavaScript (validação, eventos, LocalStorage, Fetch API) e React completo (componentes, props, useState, useEffect, Context e Router).

A API TVMaze foi uma escolha acertada por ser pública, confiável e sem chave de acesso, eliminando problemas de autenticação durante o desenvolvimento e na apresentação.

11.1 Aprendizados

- Context API evita prop drilling em projetos com múltiplas rotas.
- Hooks customizados (useLocalStorage) tornam o código mais limpo e reutilizável.
- Tailwind CSS v4 simplifica a responsividade sem Media Queries manuais.
- Efeitos assíncronos no React exigem atenção para evitar vazamentos de memória.

11.2 Sugestões de Melhorias Futuras

- Autenticação de usuários com lista pessoal na nuvem (Firebase).
- Página de temporadas e episódios usando endpoints adicionais da TVMaze.
- Sistema de avaliação por estrelas e histórico de datas de assistido.
- Modo claro/escuro com alternância pelo usuário.
- Testes automatizados com React Testing Library e Vitest.